

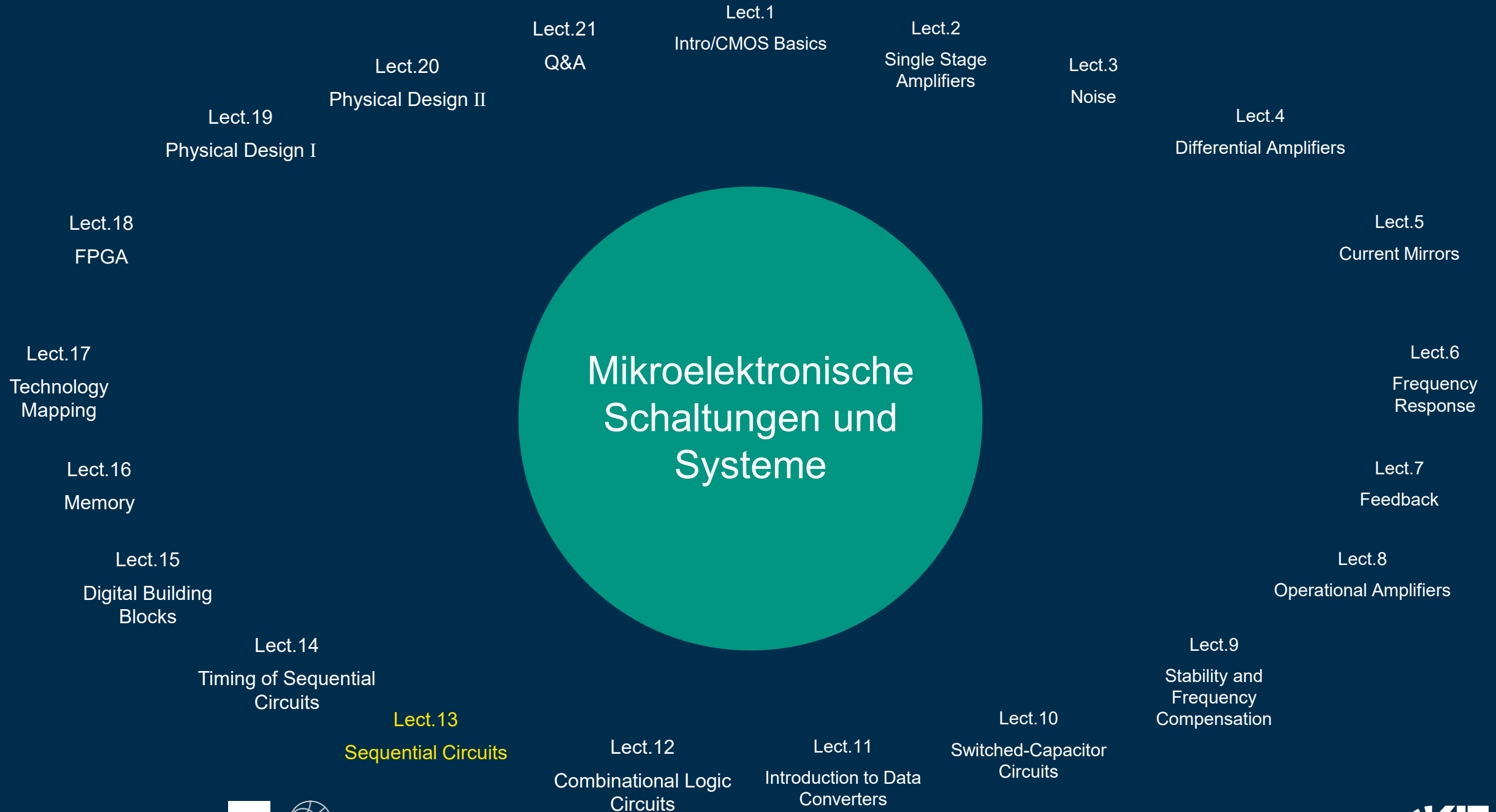
Mikroelektronische Schaltungen und Systeme

Vorlesung 13 Sequentielle Schaltungen

Prof. Jürgen Becker
Institut für Technik der Informationsverarbeitung (ITIV)



Mikroelektronische Schaltungen und Systeme



1. Latches und Flip-Flops
2. Asynchrones/Synchrones Logikdesign
3. Finite State Machines



Latches und Flip-Flops

1

Latches und FLIP-FLOPS

Bistabiles Element

- Um einen **Binärwert** zu **speichern**, wird ein **bistabiles Element** benötigt. Unten ist ein **bistabiles Element** mit **kreuzgekoppelten** (*cross-coupled*) **Invertern** dargestellt. **Kreuzgekoppelt** bedeutet, dass der **Output** eines Inverters der **Input** des **anderen Inverters** ist.



Referenz S. 110 [1]

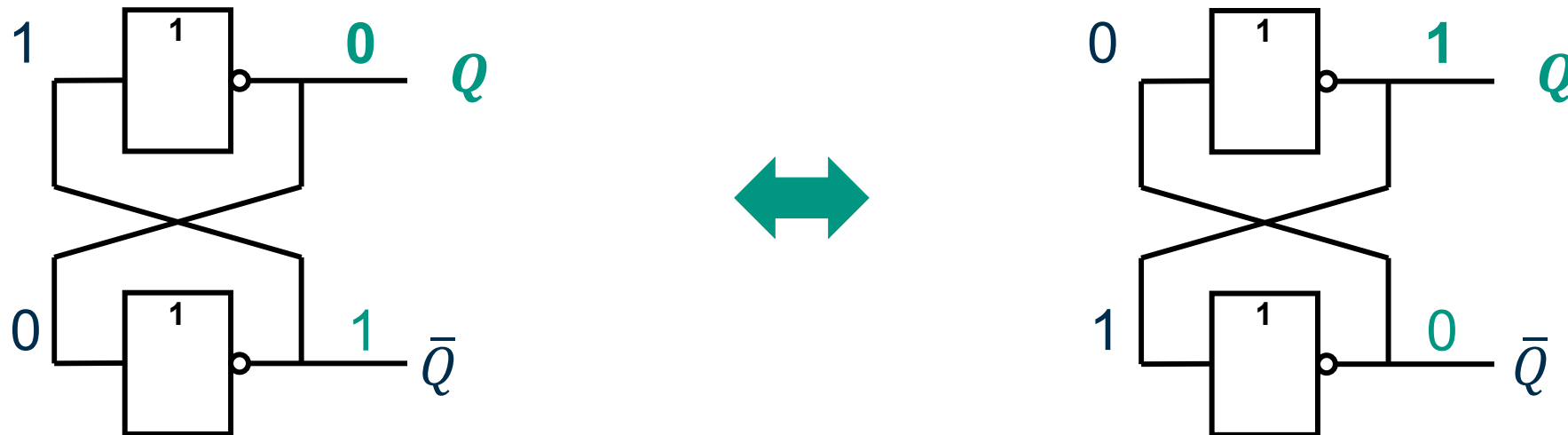
Latches und FLIP-FLOPS

Bistabiles Element

- Ein **Element** mit N stabilen Zuständen stellt $\log_2 N$ bits dar
- Ein **bistabiles Element** speichert 1 Bit

Anmerkung:

Element hat **2 Outputs**: Wert von **1 Output** reicht aus, um die Zustandsvariable anzuzeigen



Referenz S. 110 [1]

Latches und FLIP-FLOPS

Bistabiles Element

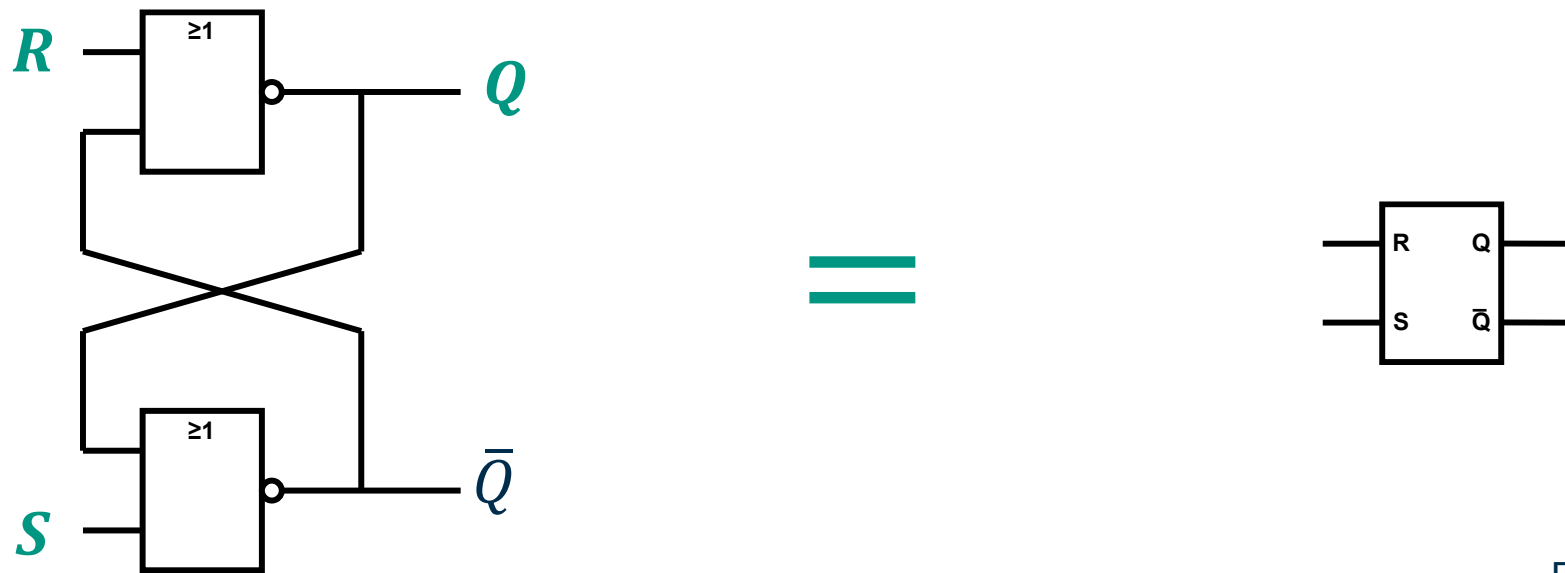
Anmerkung:

- In **Einschaltzustand**, ist der **Zustand** des *bistabilen* Elements *unbekannt* und *unvorhersehbar*:
 - > Er kann sich zwischen den Einschaltvorgängen unterscheiden
- Um den *Zustand* des Elements zu definieren, muss das Element *initialisiert* werden

Latches und FLIP-FLOPS

SR-Latch

- Um den *Zustand* von Q zu steuern:
 - Das *bistabile* Element kann aus zwei *NOR*-Gattern realisiert werden
- Der *SR-Latch* hat zwei Inputs: **Set (S)** und **Reset (R)**



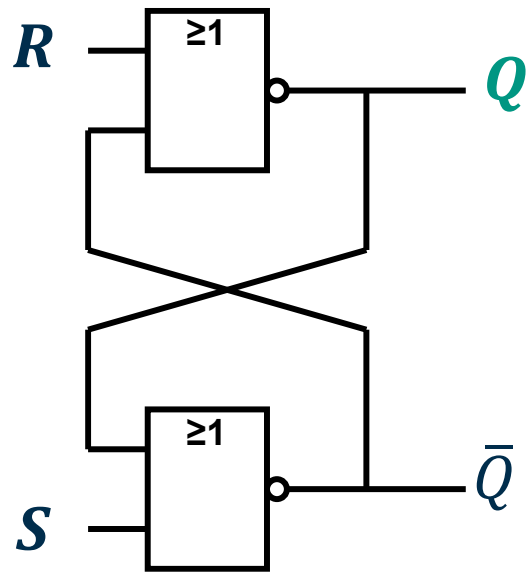
Referenz S. 111+112 [1]

Latches und FLIP-FLOPS

SR-Latch

Anmerkung:

Die **Outputs** Q und $\bar{Q}$ der **Schaltung** hängen von den **Inputs**, aber auch vom **vorherigen Zustand** der Schaltung ab (Vergleiche: **Mealy-Automat**)



R	S	Q_{prev}	$\bar{Q}_{prev}$	Q	$\bar{Q}$
0	0	-	-	Q_{prev}	$\bar{Q}_{prev}$
0	1	-	-	1	0
1	0	-	-	0	1
1	1	-	-	0	0

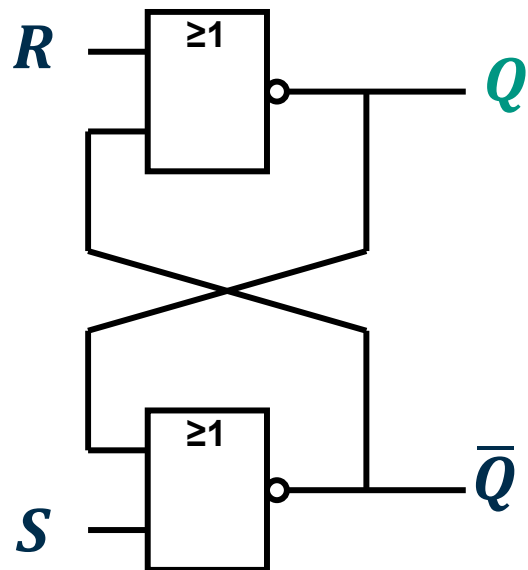
Referenz S. 111+112 [1]

Latches und FLIP-FLOPS

SR-Latch

Anmerkung:

Die **Outputs** Q und $\bar{Q}$ der Schaltung hängen von den Inputs, aber auch vom vorherigen **Zustand** der Schaltung ab (Vergleiche: *Mealy-Automat*)



Ungültige
Eingabe



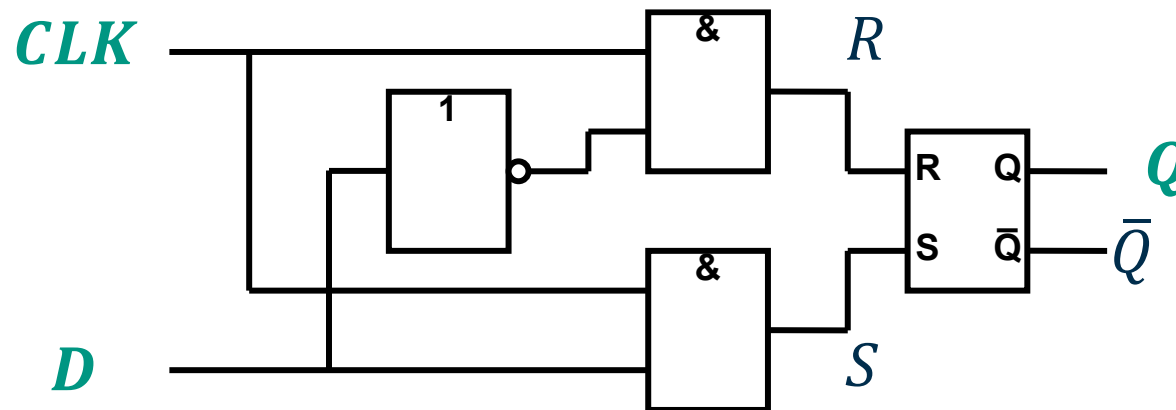
R	S	Q_{prev}	$\bar{Q}_{prev}$	Q	$\bar{Q}$
0	0	-	-	Q_{prev}	$\bar{Q}_{prev}$
0	1	-	-	1	0
1	0	-	-	0	1
1	1	-	-	0	0

Referenz S. 111+112 [1]

Latches und FLIP-FLOPS

D-Latch

- Der unten dargestellte *D-Latch* verhindert die **unzulässige Inputkonfiguration** des *RS-Latch* und führt ein:
 - einen **Clock-Input (CLK)**, um das Timing für **Zustandsänderungen** festzulegen
 - Einen **Data-Input (D)** zum Einstellen des **Zustands**
- Wenn **CLK = 1**: das *D-Latch* ist für alle **Datenflüsse** durch das Latch **transparent**



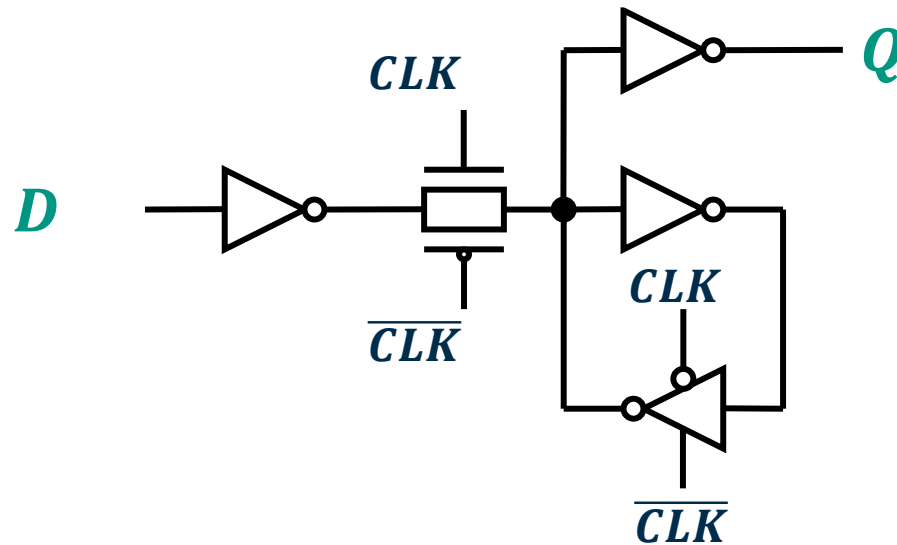
<i>CLK</i>	<i>D</i>	<i>S</i>	<i>R</i>	<i>Q</i>	$\bar{Q}$
0	-	0	0	Q_{prev}	$\bar{Q}_{prev}$
1	0	0	1	0	1
1	1	1	0	1	0

Referenz S. 113 [1]

Latches und FLIP-FLOPS

Transistor-Level-Latch

- Die folgende **Abbildung** zeigt einen **kompakten 12-Transistor-D-Latch** mit:
 - einem **Übertragungsgatter** (*transmission gate*) zum **Koppeln** & **Entkoppeln** des **Inputs**
 - drei Invertern**
 - einem **Tristate-Buffer** zum **Ansteuern** des **Outputs**

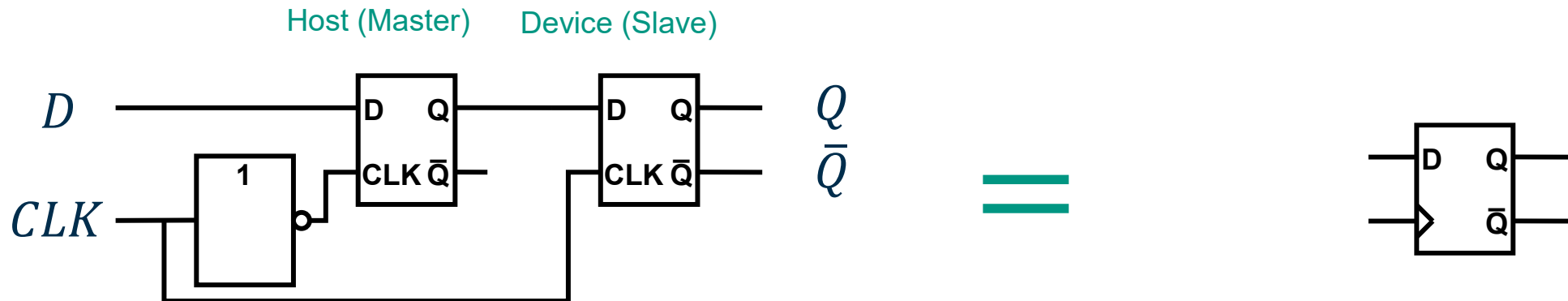


Referenz S. 117 [1]

Latches und FLIP-FLOPS

D-Flipflop

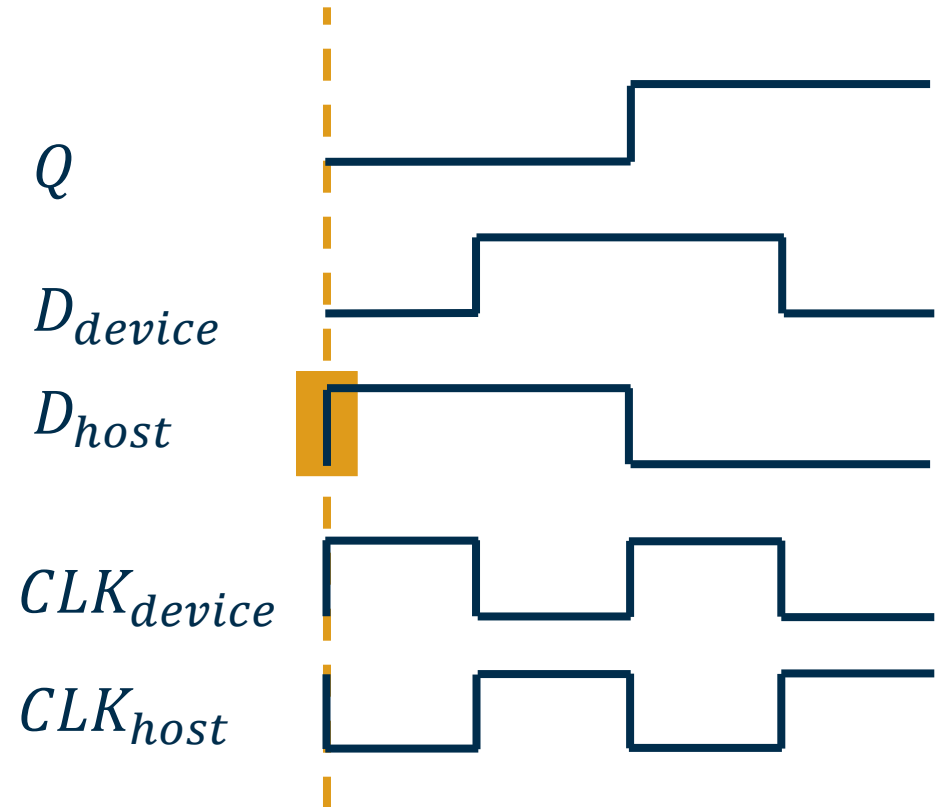
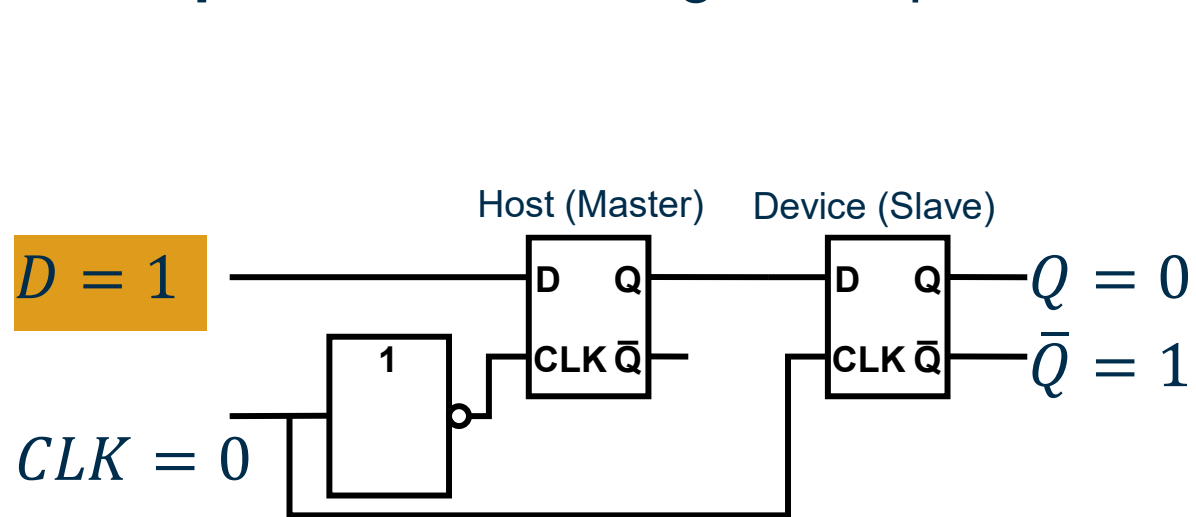
- Im Gegensatz zu einem *D-Latch* ist das *D-Flip-Flop* niemals *transparent* für den Input, sondern **legt** seinen *Zustand* und den *Output* entsprechend dem *Input* zu einem *bestimmten Zeitpunkt* fest:
 - Folgende Version: der Zeitpunkt ist die *Rising Edge* (*steigende Flanke*) des *CLK*-Inputs
 - D-Flip-Flop: - kann aus 2 *D-Latches* bestehen
- durch *komplementäre Clocks* getaktet



Latches und FLIP-FLOPS

D-Flipflop

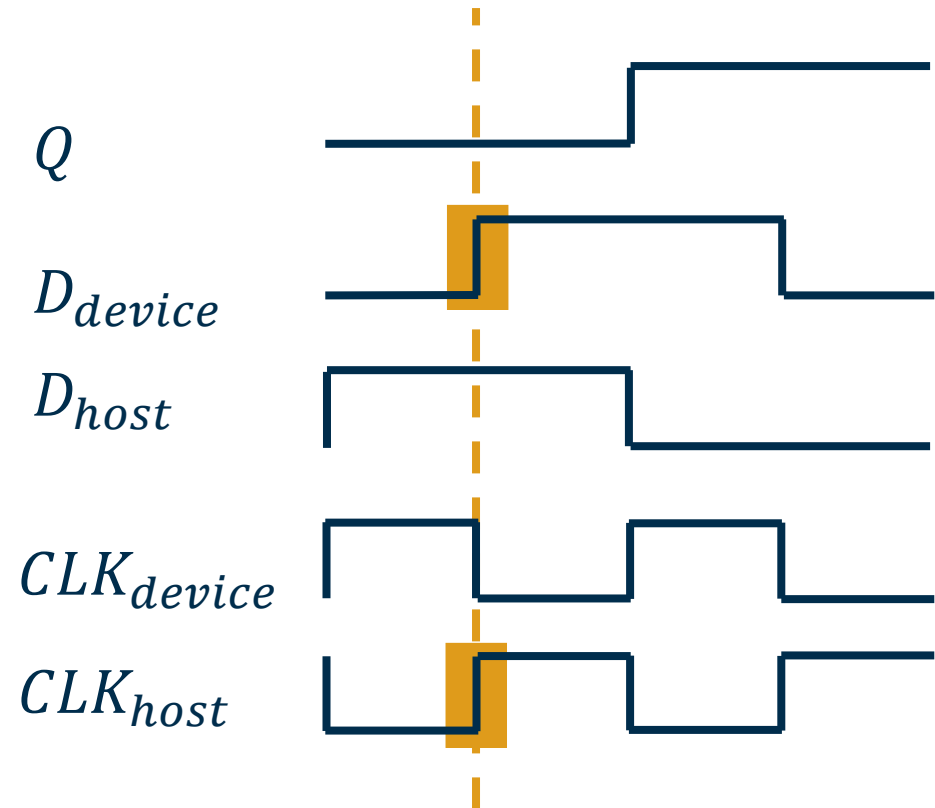
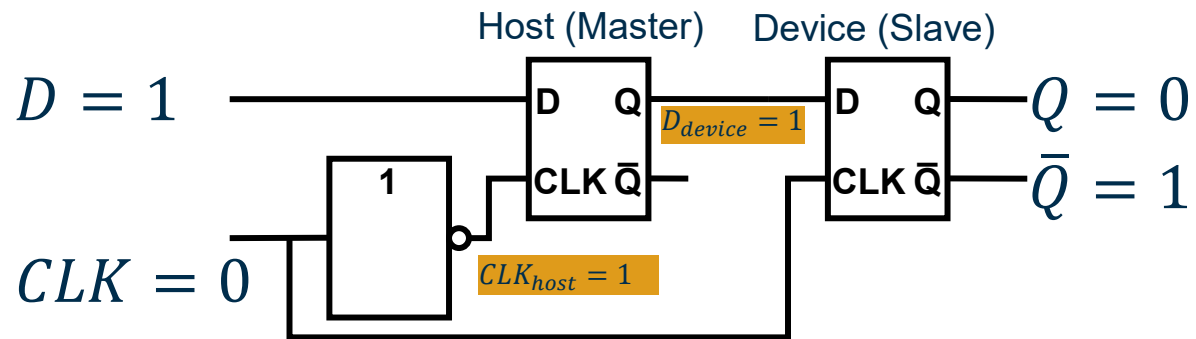
Beispiel: Weiterleitung von Input zu Output



Latches und FLIP-FLOPS

D-Flipflop

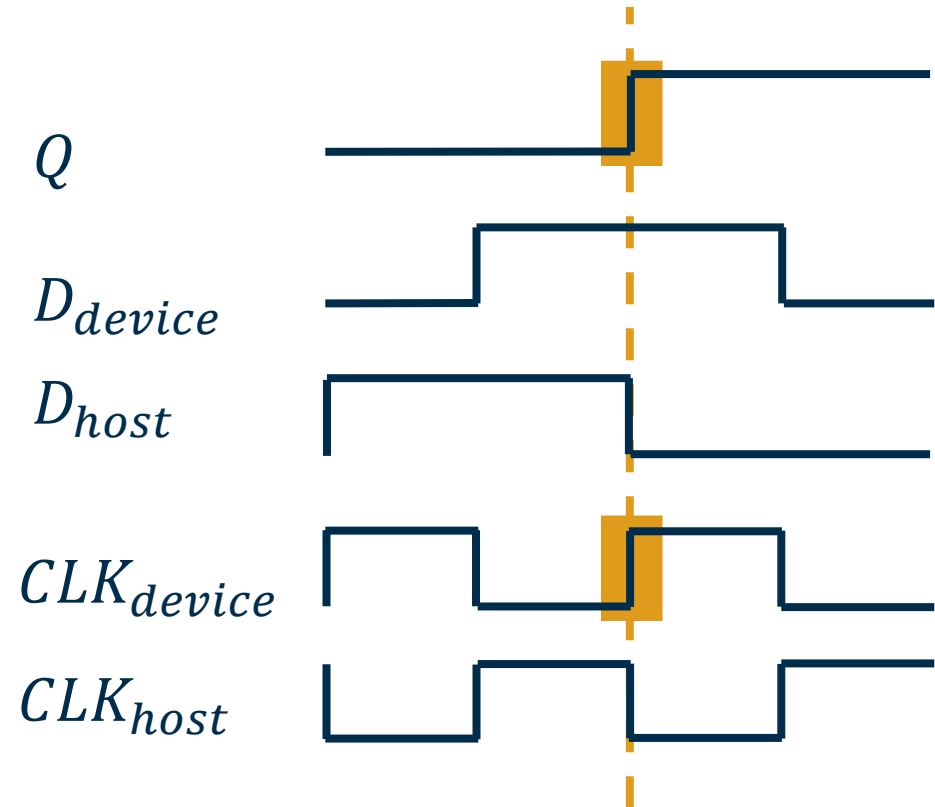
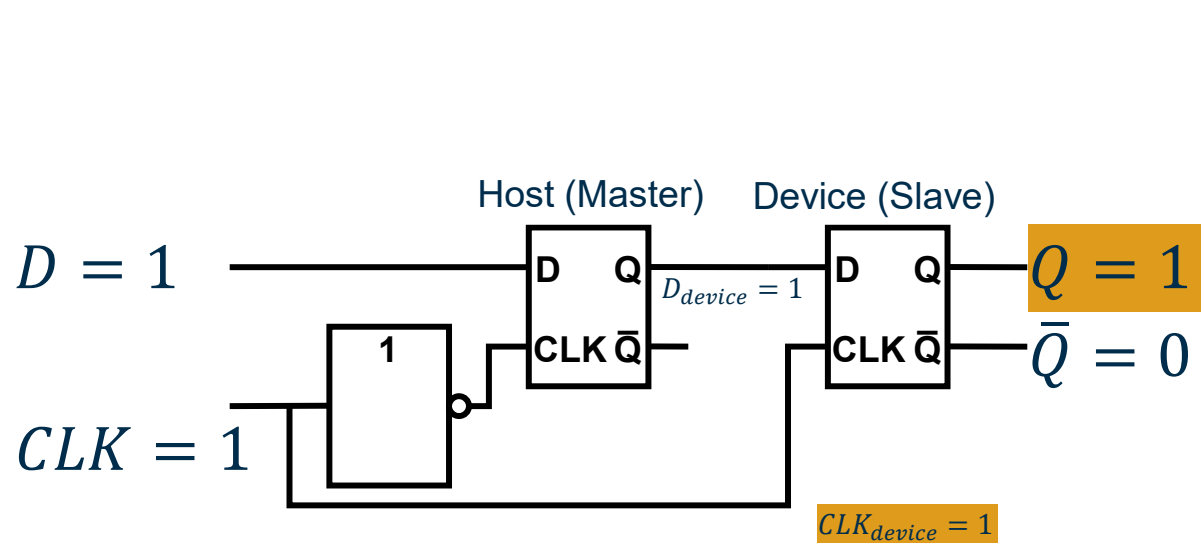
Beispiel: Weiterleitung von Input zu Output



Latches und FLIP-FLOPS

D-Flip-Flop

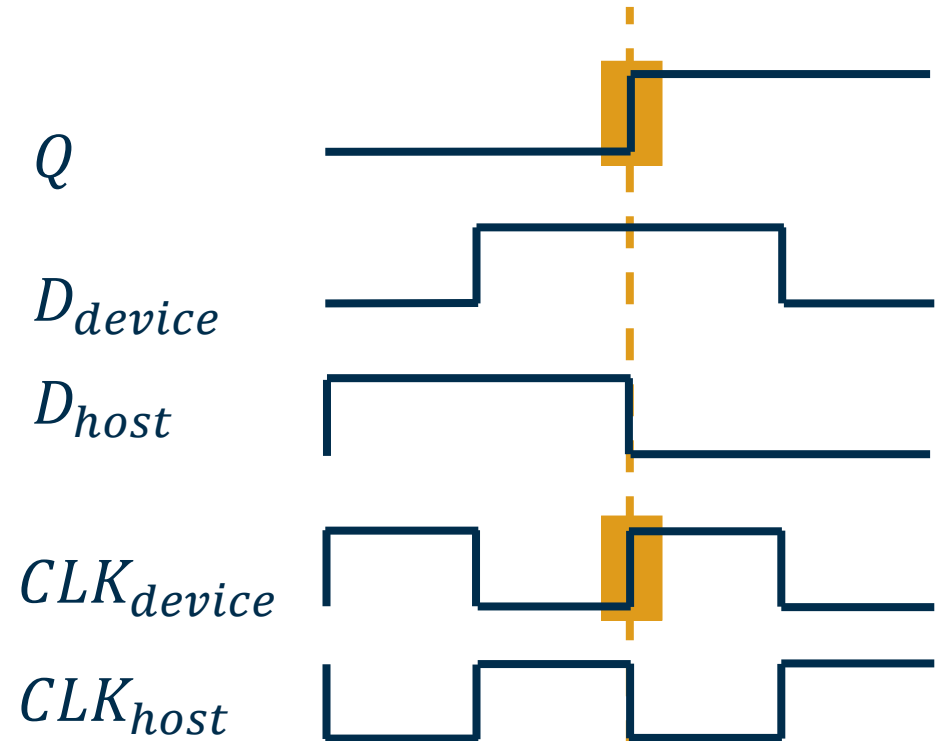
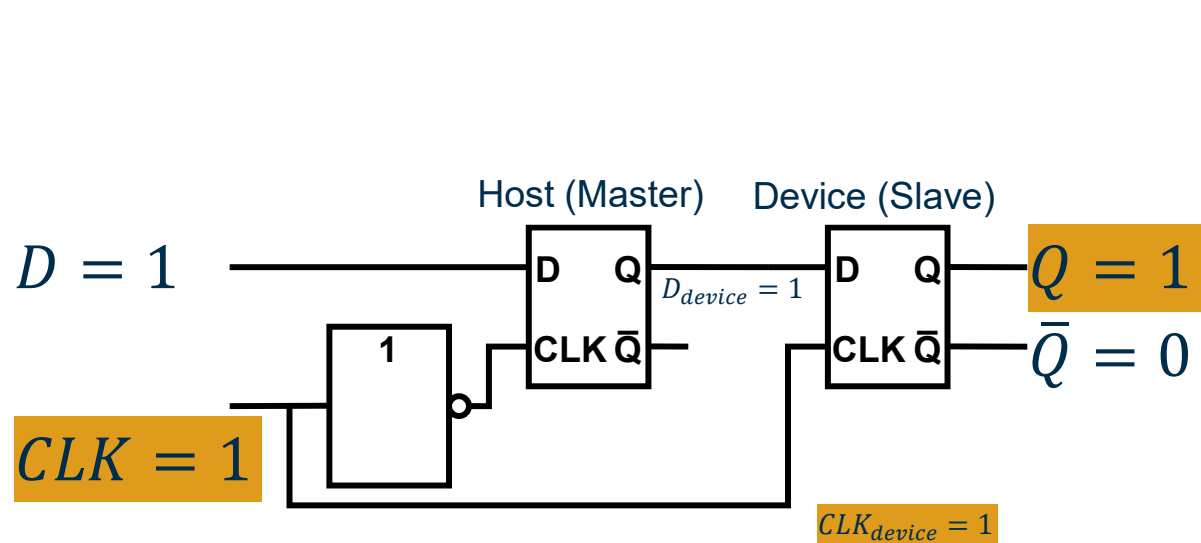
Beispiel: Weiterleitung von Input zu Output



Latches und FLIP-FLOPS

D-Flip-Flop

Beispiel: Weiterleitung von Input zu Output



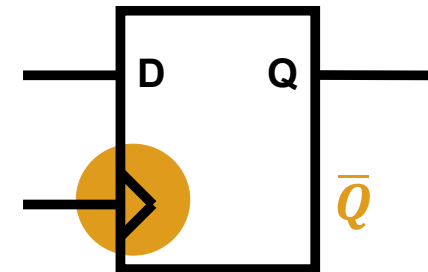
Rising Edge D-Flop-Flop

Latches und FLIP-FLOPS

D-Flip-Flop

Anmerkung:

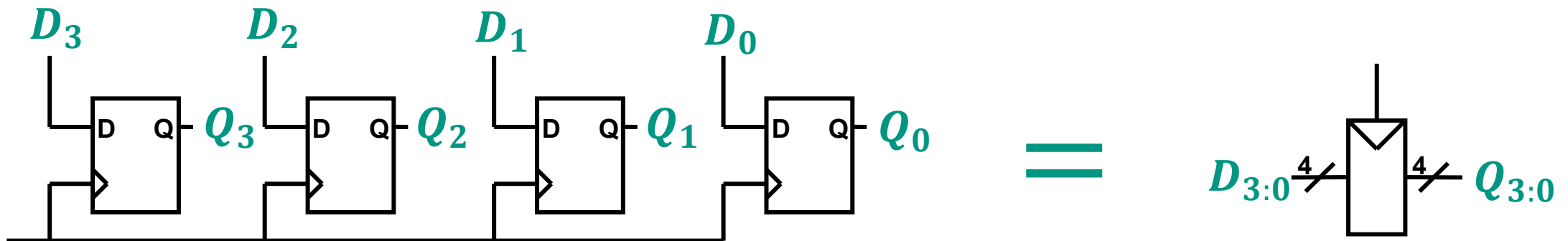
- Das *D-Flipflop* ist auch als *Master-Slave-Flipflop* oder *flanken getriggertes Flipflop* bekannt
- Das *Dreieck* in den Symbolen kennzeichnet einen *flanken getriggerten Clock-Input*
- Der *$\bar{Q}$ Output* wird oft vernachlässigt



Latches und FLIP-FLOPS

Register

- Eine **Gruppe** von N **Flip-Flops**, die mit demselben **CLK** **CLK** Signal verbunden sind, wird als **N-Bit-Register** bezeichnet
- **Register**: häufiger Baustein für die meisten **sequentiellen** Schaltungen



Latches und FLIP-FLOPS

Enabled Flipflops

- Um zu bestimmen, zu **welcher *CLK* Edge** der ***D-Input*** abgetastet werden soll, führt das ***Enabled Flip-Flop*** einen **zusätzlichen *Input*** ein: ***Enable (EN)***
- Wenn der ***Enable-Input*** auf **0** gesetzt ist, wird der ***D-Input*** bei der ***steigenden Flanke*** von ***CLK*** ***ignoriert*** und der ***Q-Output*** bleibt unverändert



Latches und FLIP-FLOPS

Resettable Flip-Flops

Um einen *Flip-Flop* auf einen *bekannten* Zustand zurückzusetzen:

- der *Resettable Flip-Flop* verfügt über einen **zusätzlichen Input**, den sogenannten *Reset-Input (RST)*





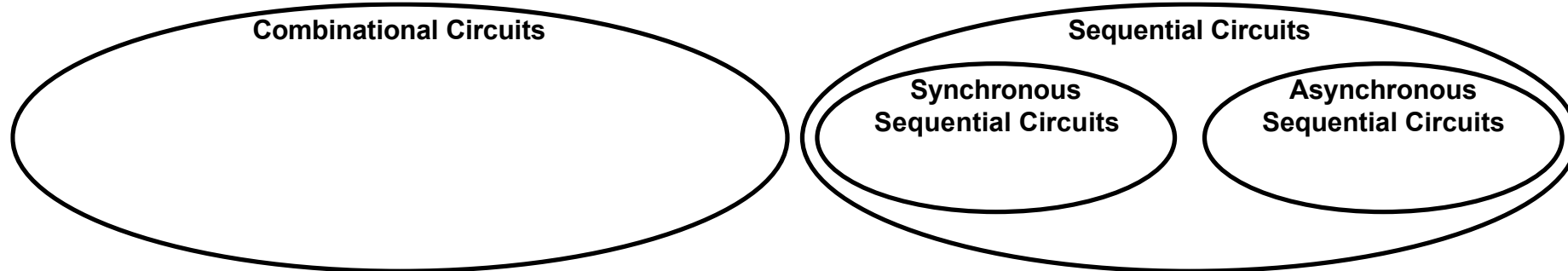
Asynchrone/Synchrone Logikdesign

2

Asynchrones/Synchrones Logikdesign

Einführung **Asynchrone** Sequentielle Schaltungen

- Im Allgemeinen umfassen **sequentielle Schaltungen** alle Schaltungen, die **keine kombinatorischen** Schaltungen sind:
 - Entwickler beschränken sich auf **synchrone sequentielle Schaltungen**, da **asynchrone Schaltungen Schwierigkeiten** mit sich bringen
 - **Asynchrone sequentielle Schaltungen** sind manchmal für *Clock-Domain-Crossings* oder bei **Eingaben zu beliebigen Zeitpunkten** erforderlich

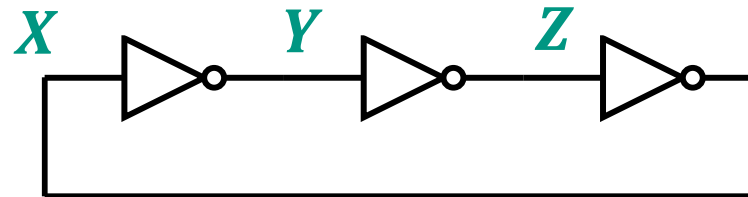


Referenz S. 119 [1]

Asynchrones/Synchrones Logikdesign

Beispiel: *Asynchrone* Schaltungen

- Der *Ringoszillator* ist eine *astabile* Schaltung, die ihre **Ausgabe** *periodisch* ändert, solange keine Inputänderung erfolgt:
 - Dies geschieht aufgrund der **Ausbreitungsverzögerungen**, die die **Inverter** enthalten, während der *Output* in die Schaltung *zurückgeführt* wird
 - Der *Ringoszillator* ist **schwer vorhersagbar**, da die **Ausbreitungsverzögerungen** von **vielen Faktoren** abhängen (Stromversorgung, Temperatur, Fertigung usw.)



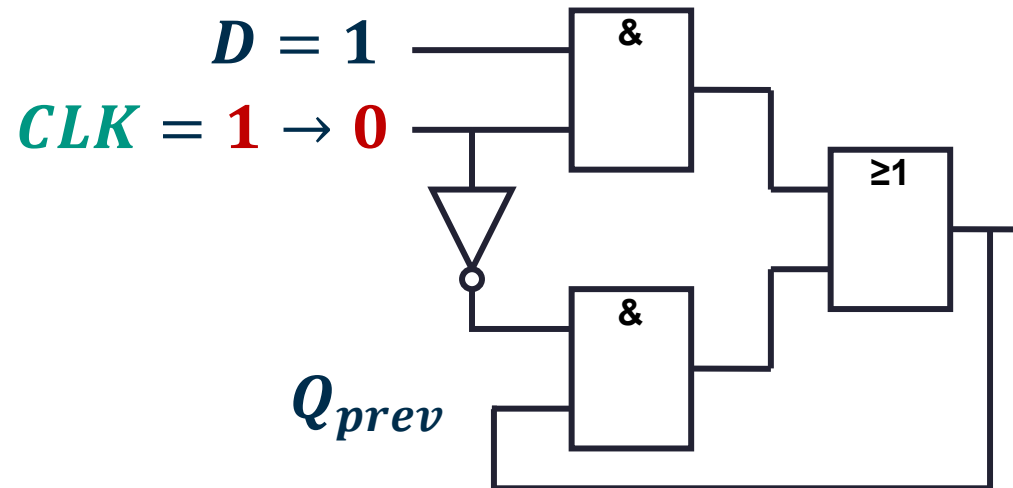
Drei-Inverter-Schleife

Referenz S. 119 [1]

Asynchrones/Synchrones Logikdesign

Beispiel: *Asynchrone* Schaltungen

- Die unten dargestellte *asynchrone* D-Latch-Implementierung weist eine *Race-Condition* auf, wenn die **Ausbreitungsverzögerung** von *CLK* durch den *Inverter länger* ist als bei den anderen Gattern:
- Der *Glitch* im kombinatorischen Teil der Schaltung führt dazu, dass die gesamte sequentielle Schaltung den Wert von *Q* nicht halten kann beim Übergang von *CLK*



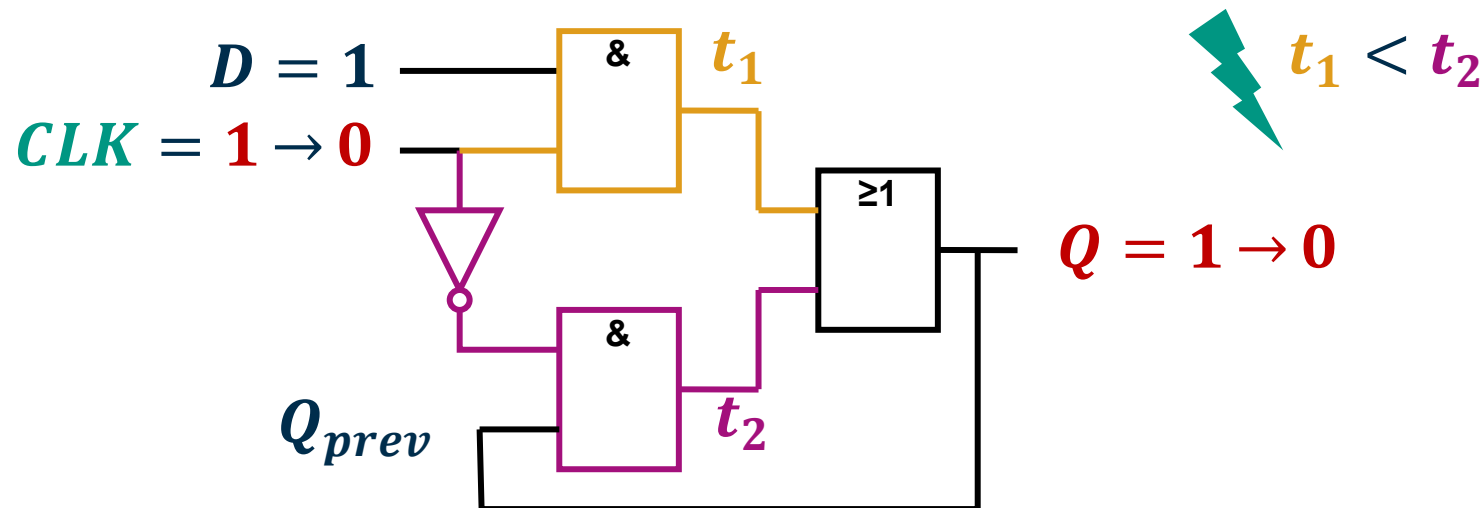
Asynchroner
D-Latch mit *Glitch*

Referenz S. 119 [1]

Asynchrones/Synchrones Logikdesign

Beispiel: *Asynchrone* Schaltungen

- Die unten dargestellte **asynchrone** D-Latch-Implementierung weist eine **Race-Condition** auf, wenn die **Ausbreitungsverzögerung** von **CLK** durch den **Inverter länger** ist als bei den anderen Gattern:
- Der **Glitch** im kombinatorischen Teil der Schaltung führt dazu, dass die **gesamte sequentielle Schaltung** den **Wert** von **Q** nicht halten kann beim **Übergang** von **CLK**



Referenz S. 119 [1]

Asynchrones/Synchrones Logikdesign

Asynchrone Schaltungen

Anmerkung:

- Die **Pfade**, die die *Ausgabe* in die *Eingabe* einspeisen, heißen **zyklische Pfade**
- Kann zu unerwünschten *Race Conditions* oder *instabilem Verhalten* führen

Asynchrones/Synchrones Logikdesign

Asynchrone Schaltungen

Anmerkung:

- Die **Pfade**, die die *Ausgabe* in die *Eingabe* einspeisen, heißen **zyklische Pfade**
 - Kann zu unerwünschten *Race Conditions* oder *instabilem Verhalten* führen
 - Kann vermieden werden, indem mit *Registern* der **Pfad** unterbrochen wird:
 - Schaltung wird umgewandelt: in Struktur aus *kombinatorischer Logik* und *Registern*
 - Die *Register* enthalten *Zustand* des Systems:
 - kann sich nur an der *Taktflanke* ändern
 - Die *kombinatorische Logik* enthält die *State-Transition Funktion* und eine *Output Funktion*

Asynchrones/Synchrones Logikdesign

Synchrone Sequenzielle Schaltungen

- Bei **synchronen sequenziellen Schaltungen** wird jeder **zyklische Pfad** mit **Registern** unterbrochen
 - Dadurch wird die **Schaltung** in eine Struktur aus **kombinatorischer Logik** und **Registern** umgewandelt
 - Die **Register** enthalten den Zustand des Systems, der sich nur an der Taktflanke ändert
 - Die **kombinatorische Logik** enthält die **State-Transition Funktion** und eine **Output Funktion**

Asynchrones/Synchrones Logikdesign

Synchrone Sequenzielle Schaltungen

Definition: *Synchrone Sequentielle* Schaltung

- Jedes Schaltungselement ist entweder ein *Register* oder eine *kombinatorische Schaltung*
 - Mindestens ein Schaltungselement ist ein *Register*
 - Alle *Register* empfangen dasselbe *Taktsignal*
 - Jeder *zyklische Pfad* enthält mindestens ein *Register*
- Gängige Vertreter: sind das *Flip-Flop* selbst, *Pipelines* und *Finite State Machines*



Finite State Machines

3

Finite State Machines

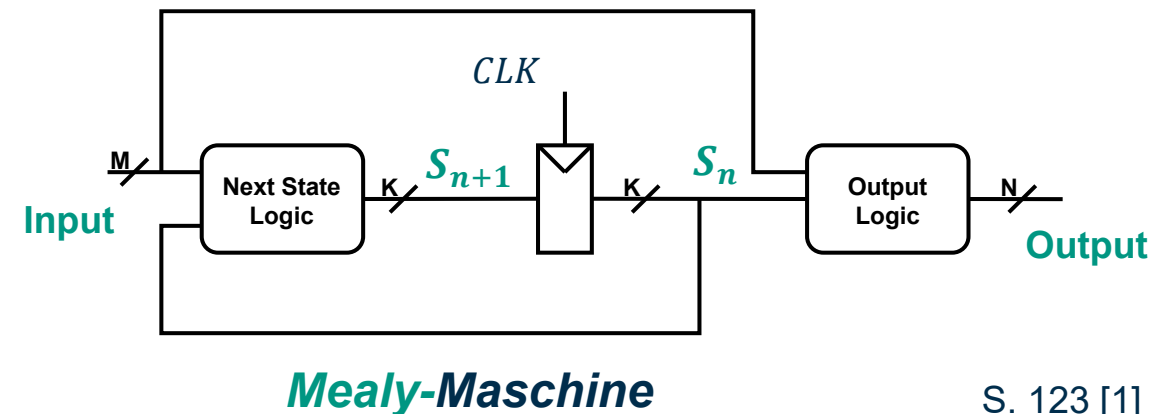
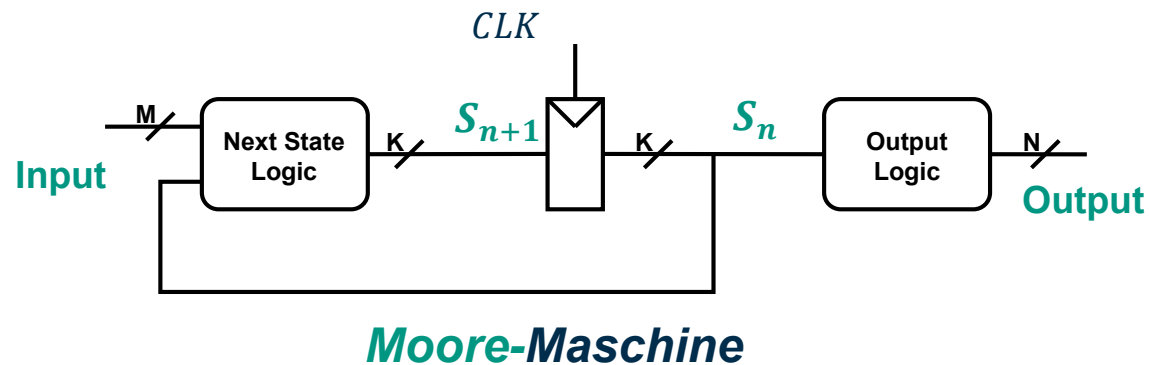
Einführung:

- Repräsentanten der synchronen sequentiellen Schaltungen sind die *Finite State Machines (FSMs)*
- Eine *FSM* besteht aus:
 - Kombinatorischer *next-state Logik* (*state transition function*),
 - Kombinatorischer *output Logik* (*output function*),
 - einem *Register* für den Zustand
- An der **Taktflanke** wechselt die *FSM* in den nächsten *Zustand*, der von der *next-state Logik* auf der Grundlage des aktuellen *Zustands* und der *Eingaben* berechnet wird
- Optional: die *FSM* empfängt ein *RESET*-Signal für seine *Register*

Finite State Machines

Moore-Maschine und Mealy-Maschine

- Die beiden allgemeinen Klassifizierungen für *FSMs* sind:
 - Moore-Maschine*
 - Mealy-Maschine*
- Die Ausgabe von *Moore-Maschinen* hängt nur vom **aktuellen Zustand** ab, die Ausgabe von *Mealy-Maschinen* hängt auch von der **aktuellen Eingabe** ab

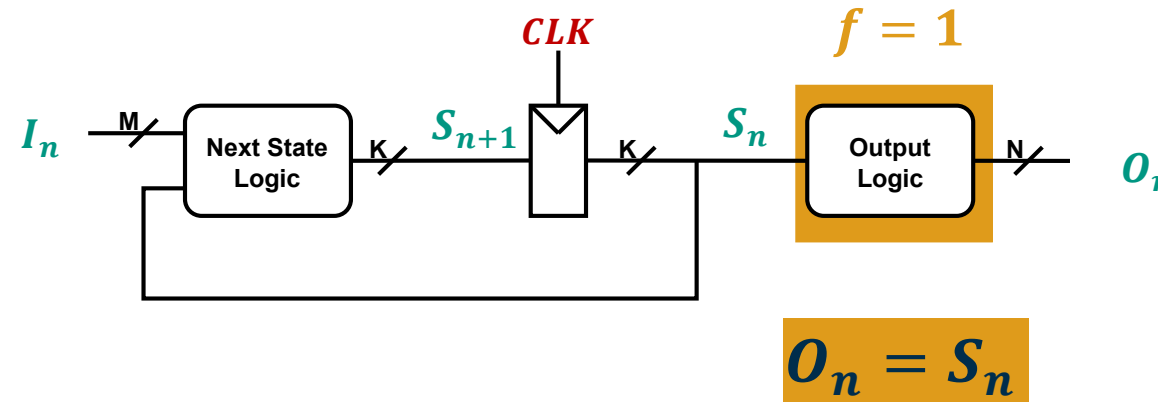


S. 123 [1]

Finite State Machines

Medwedew-Maschine

- Die **Medwedew-Maschine** ist eine spezielle **Unterklasse** der **Moore-Maschine**
- In diesem Fall ist die **output Logik** die **Identitätsfunktion** ($f = 1$), und der **output** entspricht dem **Zustandswert**



Medwedew-Maschine

Referenz S. 123 [1]

Endliche Automaten

Zustandsübergangstabelle und Ausgabetabelle

- Die **next-state** Logik kann mit einer **state transition table** definiert werden
- Die **output** Logik kann mit einer **output table** definiert werden
- Bei **Mealy-Maschine**: aufgrund **gemeinsamer** Abhängigkeiten sinnvoll, diese zu **kombinieren**

Beispiel:

Current State <i>S</i>	Input <i>A</i>	Next State <i>S'</i>
S0	0	S1
S0	1	S0
S1	0	S1
S1	1	S2
S2	0	S1
S2	1	S0

Moore state transition table

Current State <i>S</i>	Output <i>Y</i>
S0	0
S1	0
S2	1

Moore output table



Current State <i>S</i>	Input <i>A</i>	Next State <i>S'</i>	Output <i>Y</i>
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

Mealy state transition & output table

Referenz S. 134 [1]

Endliche Automaten

Zustandsübergangstabelle und Ausgabetabelle

- Die **next-state** Logik kann mit einer **state transition table** definiert werden
- Die **output** Logik kann mit einer **output table** definiert werden
- Bei **Mealy-Maschine**: aufgrund **gemeinsamer** Abhängigkeiten sinnvoll, diese zu **kombinieren**

Beispiel:

Current State <i>S</i>	Input <i>A</i>	Next State <i>S'</i>
S0	0	S1
S0	1	S0
S1	0	S1
S1	1	S2
S2	0	S1
S2	1	S0

Moore state transition table

S2 bei Mealy nicht notwendig, da nur Übergangs-state.

Current State <i>S</i>	Output <i>Y</i>
S0	0
S1	0
S2	1

Moore output table



Current State <i>S</i>	Input <i>A</i>	Next State <i>S'</i>	Output <i>Y</i>
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

Mealy state transition & output table

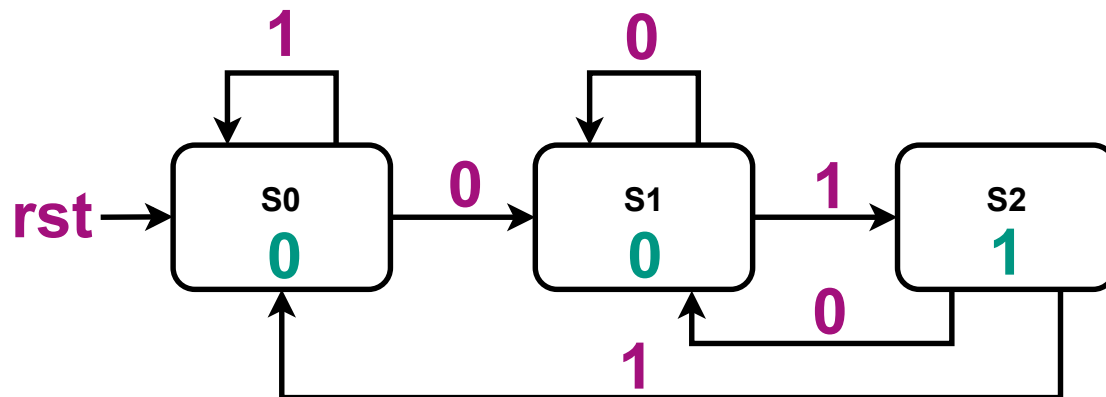
Referenz S. 134 [1]

Finite State Machines

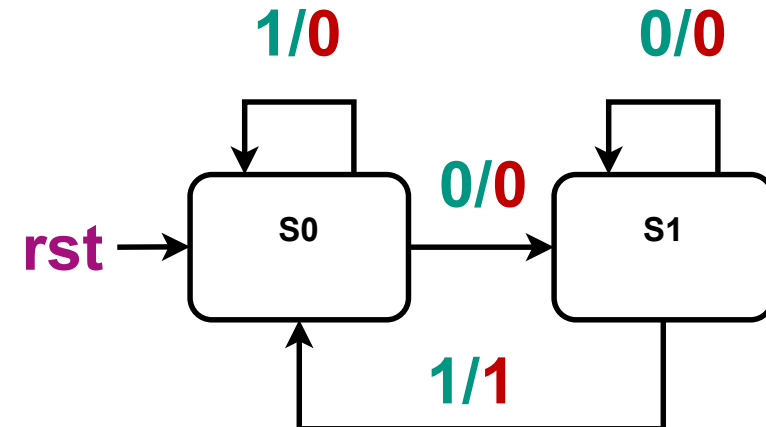
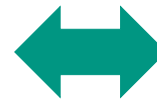
Zustandsübergangsdiagramme

- Zustandsübergangstabelle: grafisch durch ein **State-Transition** Diagramm dargestellt
- Bei **Moore-Maschinen** werden die **Outputs** in den **States** beschriftet.
- Bei **Mealy-Maschinen** werden die **Outputs** auf den **Transitions** gekennzeichnet {**State/Output**}

Beispiel:



Moore-Maschine



Mealy-Maschine

Referenz S. 133 [1]

Endliche Automaten

Zustandskodierung

- Unterschiedliche **Zustandskodierungen** führen zu unterschiedlichen Designs und Schaltungen:
 - Es ist möglich, eine gute Kodierung zu wählen: indem die **Zustände** so umstrukturiert werden, dass verwandte **Zustände** Bits in der **Kodierung** gemeinsam nutzen
- Gängige **Kodierungen** für **K Zustände** sind:
 - **Binäre Kodierung** mit $n = \lceil \log_2 K \rceil$ bit
 - **One-Hot-Kodierung** mit $n = K$ bit

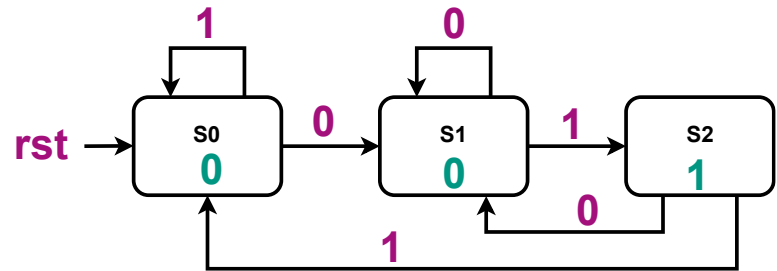
State	Encoding $S_{1:0}$
S0	00
S1	01
S2	10

State	Encoding $S_{2:0}$
S0	001
S1	010
S2	100

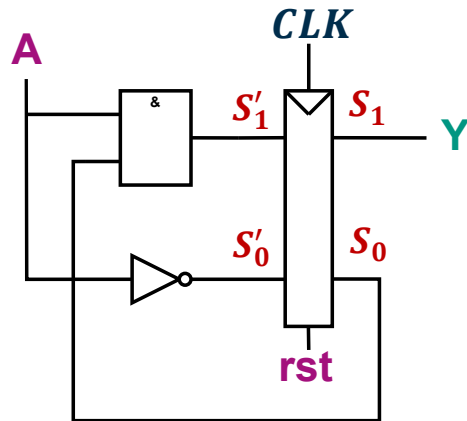
Endliche Automaten

Von State-Transition Definition zur Logik

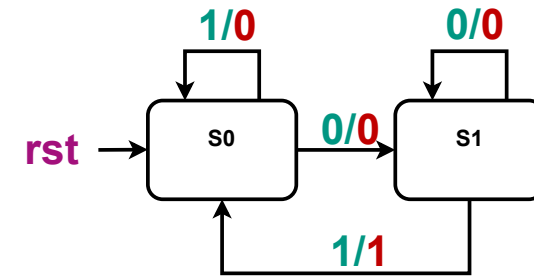
Beispiel: *Zustandsübergangsdiagramm* zur Logik mit *binärer Kodierung*



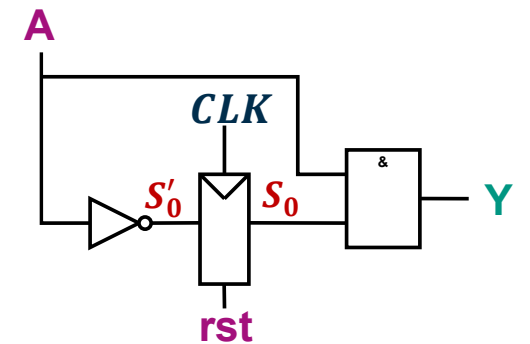
State	Encoding $S_{1:0}$
S0	00
S1	01
S2	10



Moore-Maschine



State	Encoding S_0
S0	0
S1	1



Mealy-Maschine

Referenz S. 135 [1]

Endliche Automaten

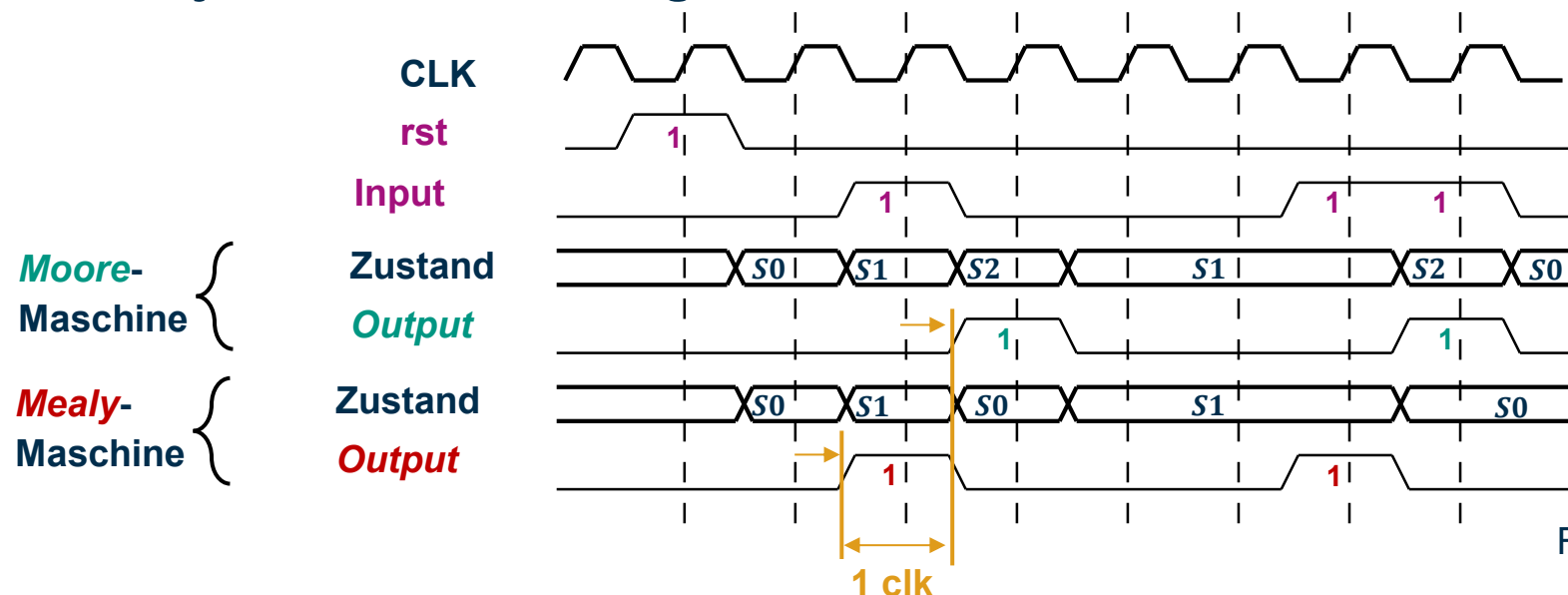
Timing: *Moore*- vs. *Mealy*-Maschine

Anmerkung:

Wenn *Moore*/*Mealy*-Maschinen voneinander abgeleitet sind:

- > *Mealy*-Maschine reagiert einen **Clockzyklus früher**: **Output kombinatorisch** mit **Input** verbunden
- > wenn *Mealy*-Output durch ein **Register** verzögert: **Übereinstimmung** mit *Moore*-Output
- > Beide **Zeitverhalten** sind je nach **Anwendungsfall** vorteilhaft

Beispiel:



Referenz S. 135 [1]

Finite State Machines

Logik zu *FSM*

- Vorgehensweise zur Ableitung des *State-Transition* Diagramms aus dem **Schaltplan**:
 1. Bestimmen Sie *Inputs*, *Outputs* und *Statebits* (*Register*)
 2. Bestimmen Sie die *state-transition* Funktion und die *output* Funktion
 3. Erstellen Sie eine *state-transition*- und *output* Tabelle
 4. Eliminieren Sie **nicht erreichbare Zustände**: können niemals aktueller Zustand sein
 5. Weisen Sie jeder gültigen Zustandsbitkombination eine *Kennung* zu
 6. Zeichnen Sie ein *State-Transition* Diagramm

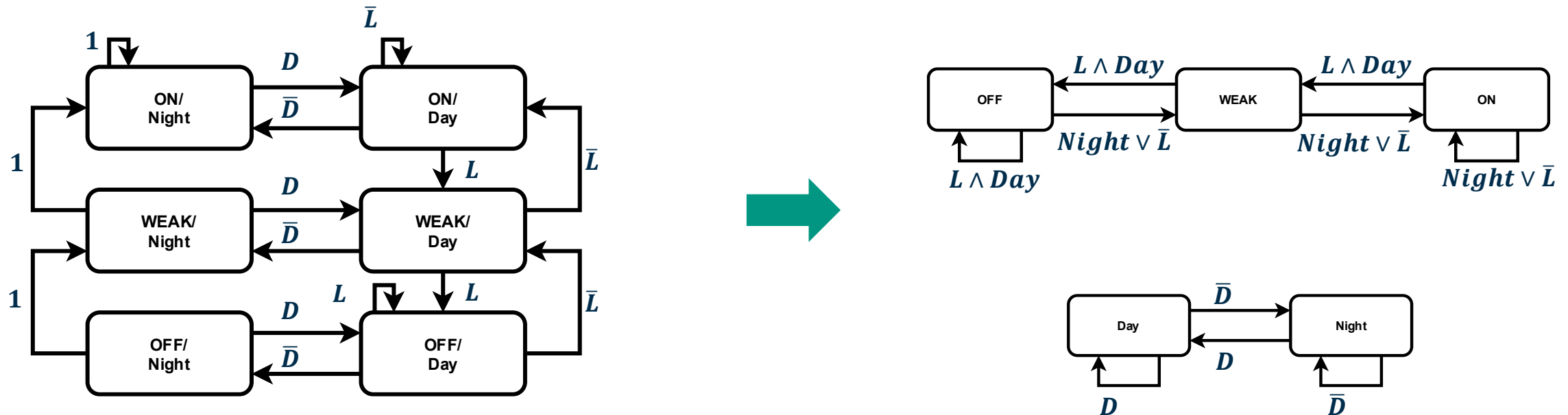
Siehe S. 137 [1]

Finite State Machines

Factoring Finite State Machines

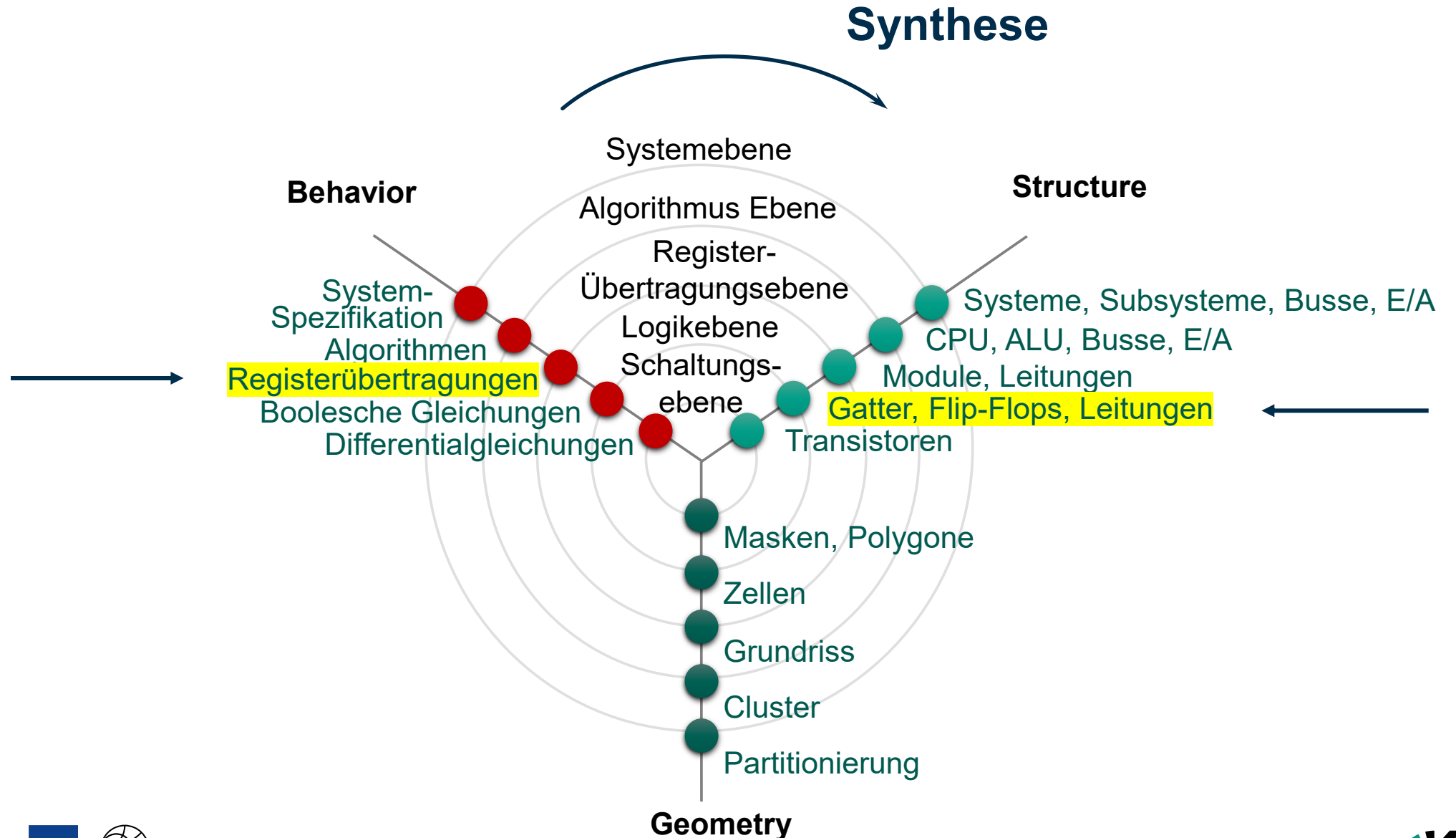
- **Factoring:** Anwendung von **Modularität** & **Hierarchie** auf **Zustandsmaschinen**:
 - Vorteilhaft für die Entwicklung komplexer **FSMs**
 - Die **FSM** wird **aufgeteilt**: **Ausgabe** einer **FSM** die **Eingabe** einer anderen ist

Beispiel: Straßenlaterne mit Licht- und Tag-/Nacht-Abhängigkeit



Advance Organizer

Y-Diagramm



Referenz

- [1] S. L. Harris und D. M. Harris, *Digital design and computer architecture*, ARM® edition. Waltham, MA: Morgan Kaufmann, 2016.

